Human-AI Lab Platform — System Design

A new website and content dashboard for the Human-Centered AI Research Lab, Department of Computer Science and Engineering, University of Moratuwa.

Author	Daham Pathiraja
Email	daham.23@cse.mrt.ac.lk

Document	System Design Document
Version	0.3 · Draft for review
Date	27 September 2026
Frontend	React 19 · TypeScript
Backend	TypeScript on Cloudflare Workers
Cloud	Cloudflare (global network) · free plan
Running cost	$0 / month
Status	Platform live on Cloudflare; public site still served from Netlify until cut-over

> **What changed since version 0.2.** Version 0.2 planned the platform on AWS (Java on Lambda, CloudFront, DynamoDB, Cognito) at about $65–75 a month. The platform has since been built and deployed on **Cloudflare's free plan** instead, with the Content API rewritten in TypeScript. It keeps the same design idea, drafts published as frozen releases, and the same API contract, at no running cost. The Java service is kept in the repository for reference but is not deployed.

Summary

Today the lab website runs on Netlify and is edited by changing code: every update needs a developer, a commit and a redeploy, and many images are hotlinked from third-party sites that can vanish at any time.

The new platform lets **invited lab members update the website themselves** through a secure dashboard, while the public site stays **fast, free and always online**.

> **The one idea to remember.** Editors work on *drafts*. Nothing reaches visitors until an Admin clicks **Publish**, which saves a complete, frozen snapshot of the site (a *release*). Visitors only ever read these snapshots, never the drafts. So a problem in the dashboard cannot take the public site down, and any mistake can be undone in seconds by switching back to an earlier release.

Targets at a glance

Goal	Target
Page load (mid-range phone, 4G, Sri Lanka)	under 2.5 s
Public site availability	≥ 99.9 %, unaffected if the dashboard is down
Time for a publish to go live	on the next page load (seconds)
Undo a bad publish	under 1 minute
Dashboard security	invited people only; sign-in with a code emailed to them
Running cost	$0 a month, capped by the free plan

---

1. Scope

In scope: the public website, the admin dashboard (content, photos, publishing, rollback), the Content API behind it, and the Cloudflare hosting, deployment and monitoring.

Not in this version: public user accounts or comments, search across publications, newsletters, and server-side rendering (planned, see §9). Video uploads, page slots, user management inside the dashboard and the audit log are designed but not built yet; they appear greyed out and marked *planned* in the figures.

---

2. Users and what they can do

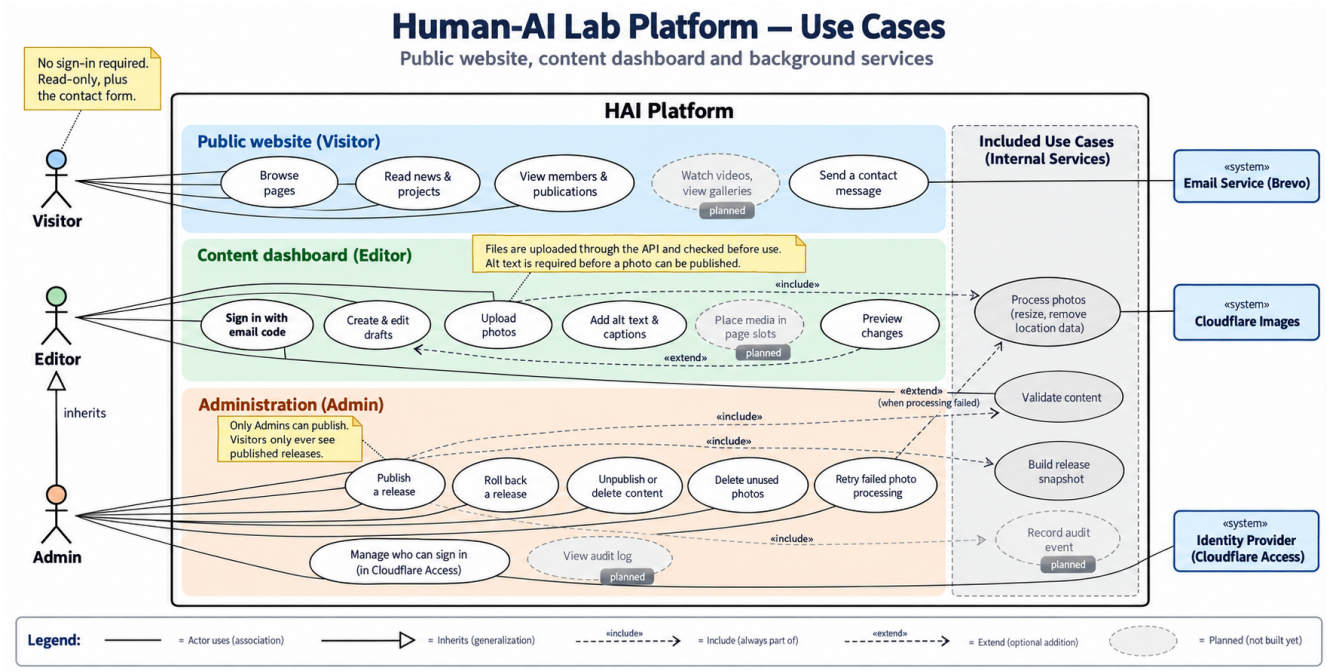


Figure 1: Use case diagram

Figure 1 — Use cases. Admins can do everything Editors can, plus publishing and administration. Greyed-out use cases are planned.

Role	Who	Can do
Visitor	Anyone	Read the website; send a contact message. No sign-in.
Editor	Invited lab members	Create and edit drafts, upload photos, add alt text, preview.
Admin	At least two senior members	Everything an Editor can, plus **publish**, **roll back**, and delete content and photos.

Who can sign in at all is set in **Cloudflare Access**: only the email addresses on its list receive a sign-in code. Who is an Admin is set by an admin list kept as a secret in the deployment. Permissions are enforced by the API on every request, not just hidden in the dashboard.

---

3. Architecture

Figure 2 — High-level architecture. Teal arrows are the visitor path; dark arrows are editing.

The system has two sides joined by exactly one link, **publishing**:

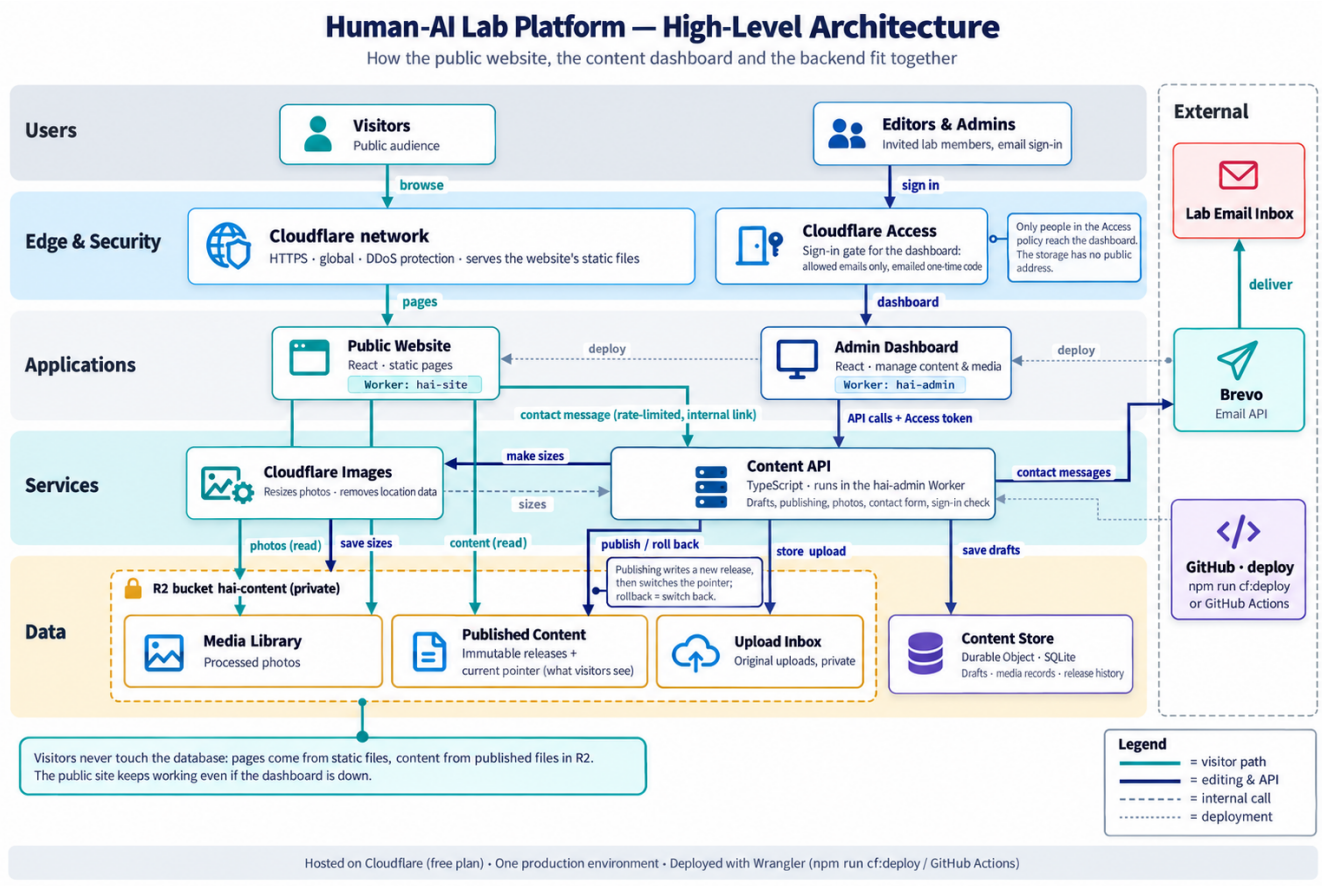


Figure 2: High-level architecture

Part	What it is	Why
Public Website	Static React app, served by the `hai-site` Worker	Pages are plain files on Cloudflare's network: fast everywhere, free, nothing to fail
Admin Dashboard	Separate React app, served by the `hai-admin` Worker at its own address	Isolated from the public site, behind its own sign-in gate
Content API	TypeScript, running in the `hai-admin` Worker	Drafts, photos, publishing, rollback, contact form, sign-in check
Content Store	A Cloudflare Durable Object with a SQLite database	Drafts, photo records and release history, in one place, so saves and publishes are exact
Published files and photos	One private R2 storage bucket	Releases (what visitors see), processed photos and original uploads
Photo processing	Cloudflare Images	Makes web-sized copies of every photo and removes location data
Identity	Cloudflare Access	Invite-only sign-in with a code emailed to the person
Email	Brevo (email service)	Delivers contact-form messages to the lab inbox

All traffic arrives through Cloudflare's network. The storage has **no public address**: only the two Workers can reach it.

---

4. Information model

Figure 3 — Domain model. Greyed-out classes are planned.

- **Content items**: news, projects, members, publications, research themes, galleries, page sections and site settings. Each is a draft until published.
- **Media assets**: uploaded photos, each with processed versions (*renditions*). Alt text is required before publishing, and photos in use cannot be deleted.
- **Releases**: frozen snapshots of the whole site. Exactly one is *current*.
- **Users**: people are identified by the email they sign in with; their role comes from the admin list.
- *Planned:* **slots and placements** (named spots on the site, such as the *homepage gallery*, and the media placed in them) and **audit events** (who did what, and when).

---

5. How it works: key flows

Each flow below is a sequence diagram; the note at its foot states the guarantee it provides.

5.1 A visitor views a page

Figure 4 — Pages are static files; content comes from published files in storage, read by a small Worker. No database on this path.

5.2 An editor signs in

Figure 5 — Only invited people receive a sign-in code, and there are no passwords. The API checks the sign-in and the role again on every request.

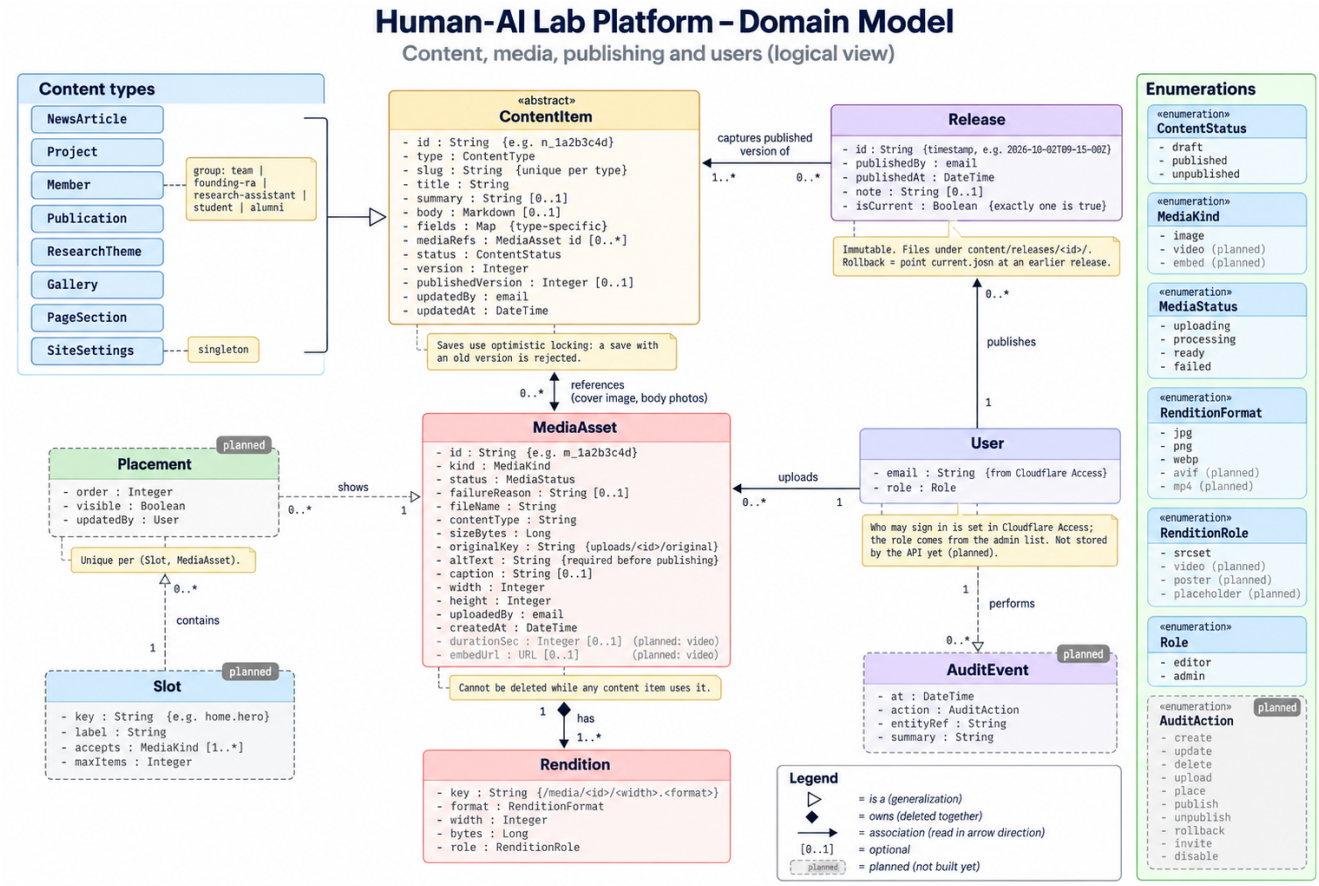


Figure 3: Domain model

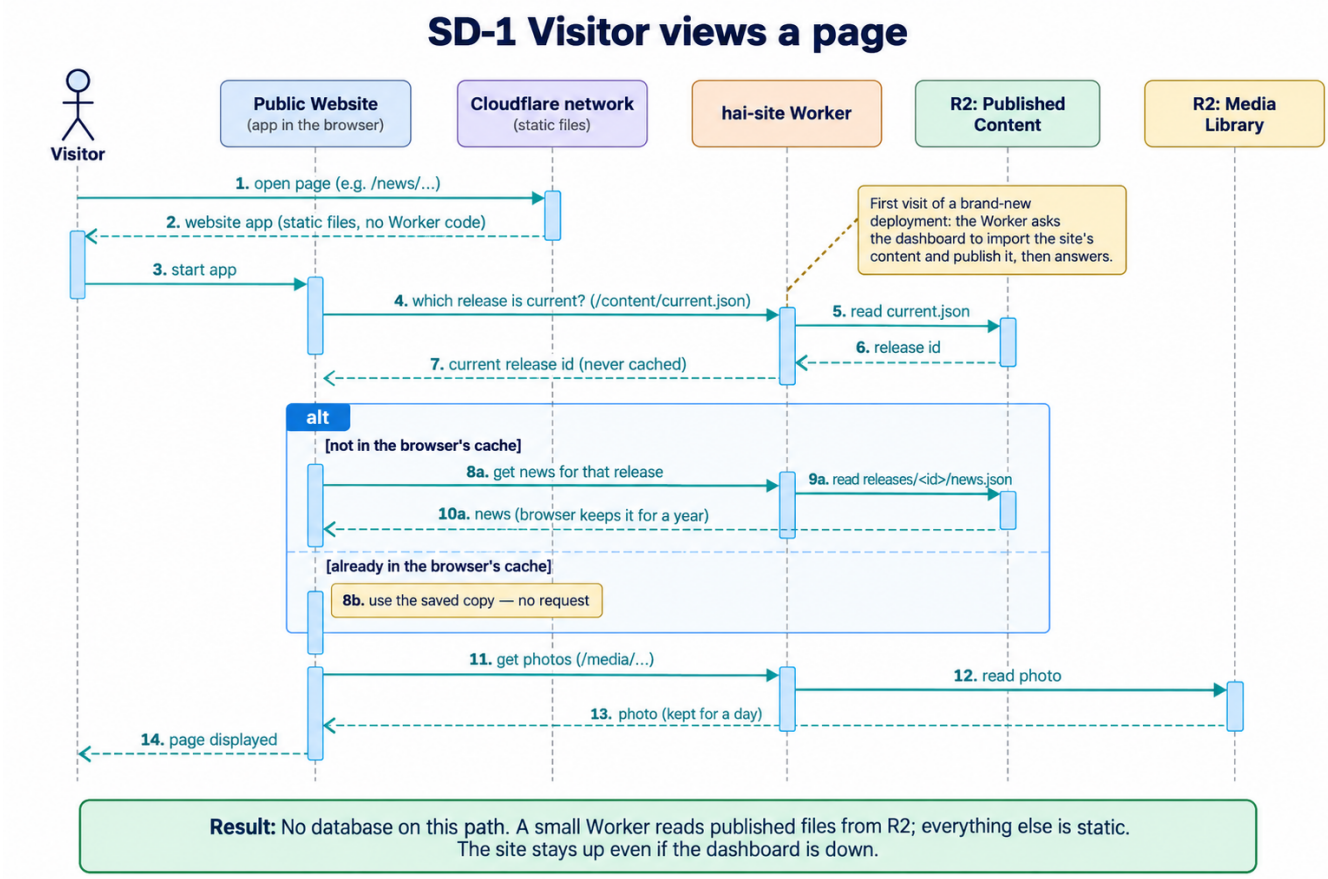


Figure 4: SD-1 Visitor views a page

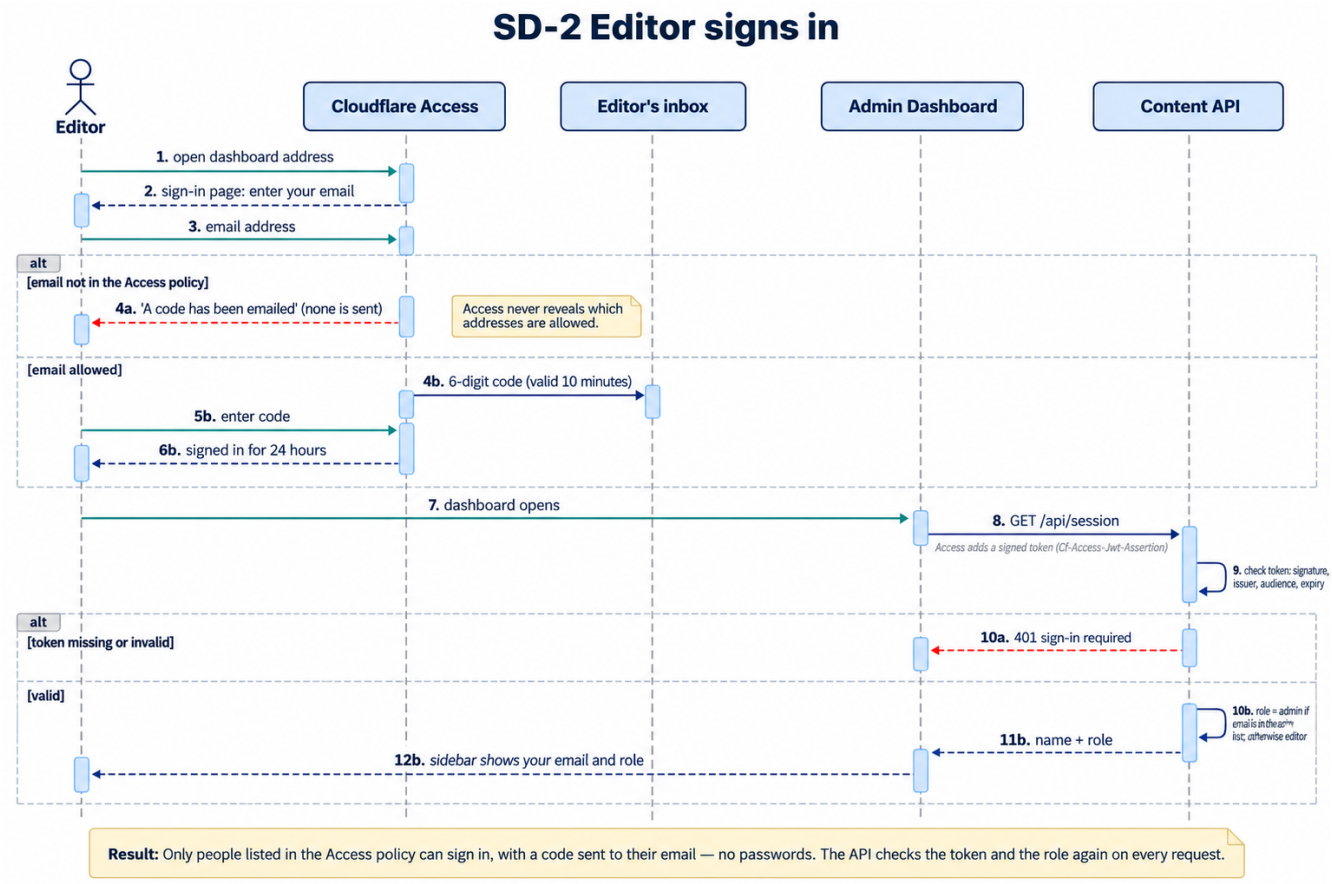


Figure 5: SD-2 Editor signs in

5.3 An editor uploads a photo

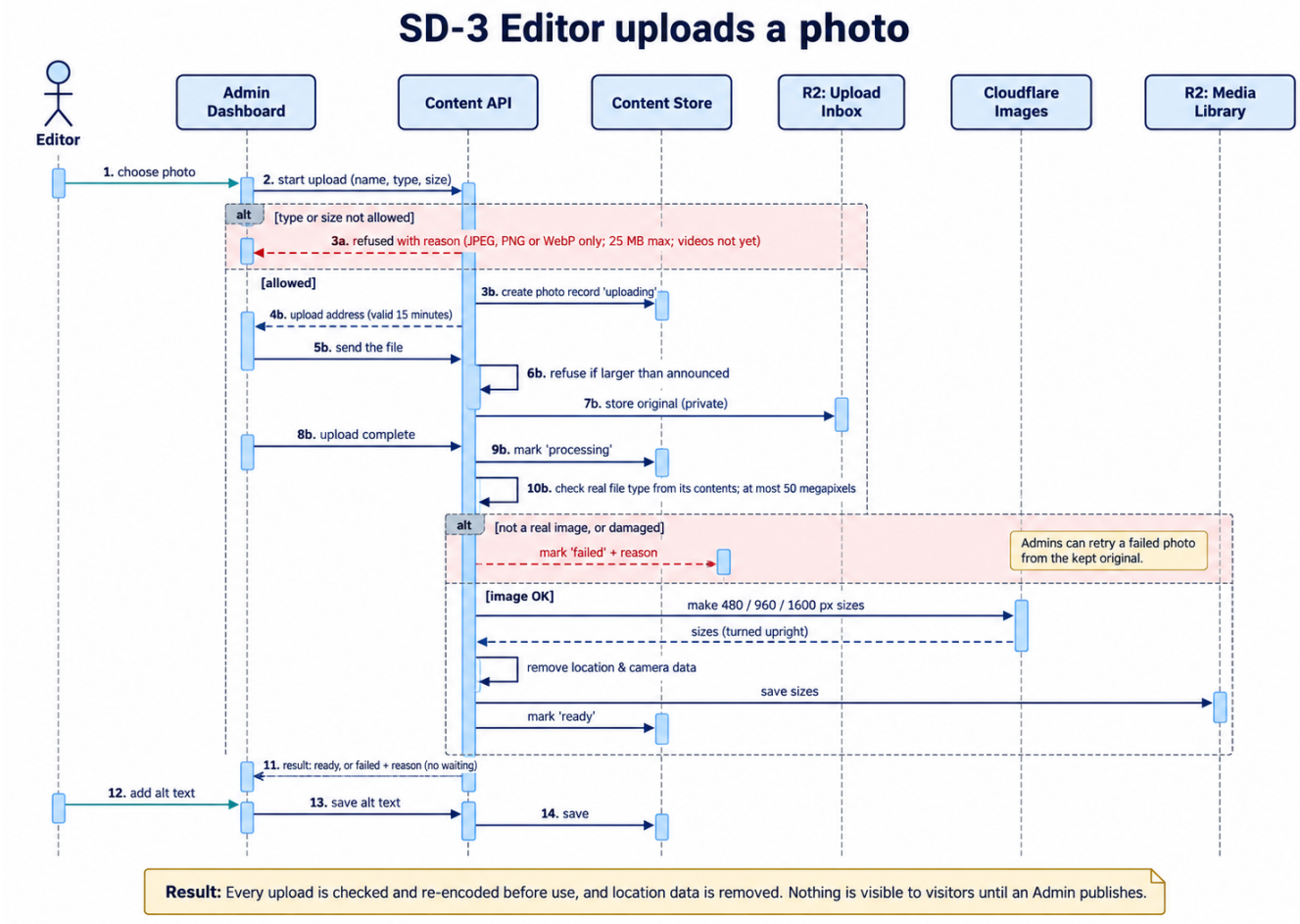


Figure 6: SD-3 Editor uploads a photo

Figure 6 — Every upload is checked and re-encoded before use, and location data is removed. Nothing is visible to visitors until an Admin publishes.

5.4 An admin publishes a release

Figure 7 — Content is validated, the new release is written in full, then switched on in one step. Visitors never see a half-published site.

5.5 An admin rolls back

Figure 8 — Rollback only moves the "current" pointer. It takes seconds and leaves drafts untouched.

5.6 A visitor sends a contact message

Figure 9 — Rate-limited and spam-checked. Messages are emailed to the lab, not stored.

---

6. Deployment on Cloudflare

Figure 10 — Production deployment on Cloudflare's free plan.

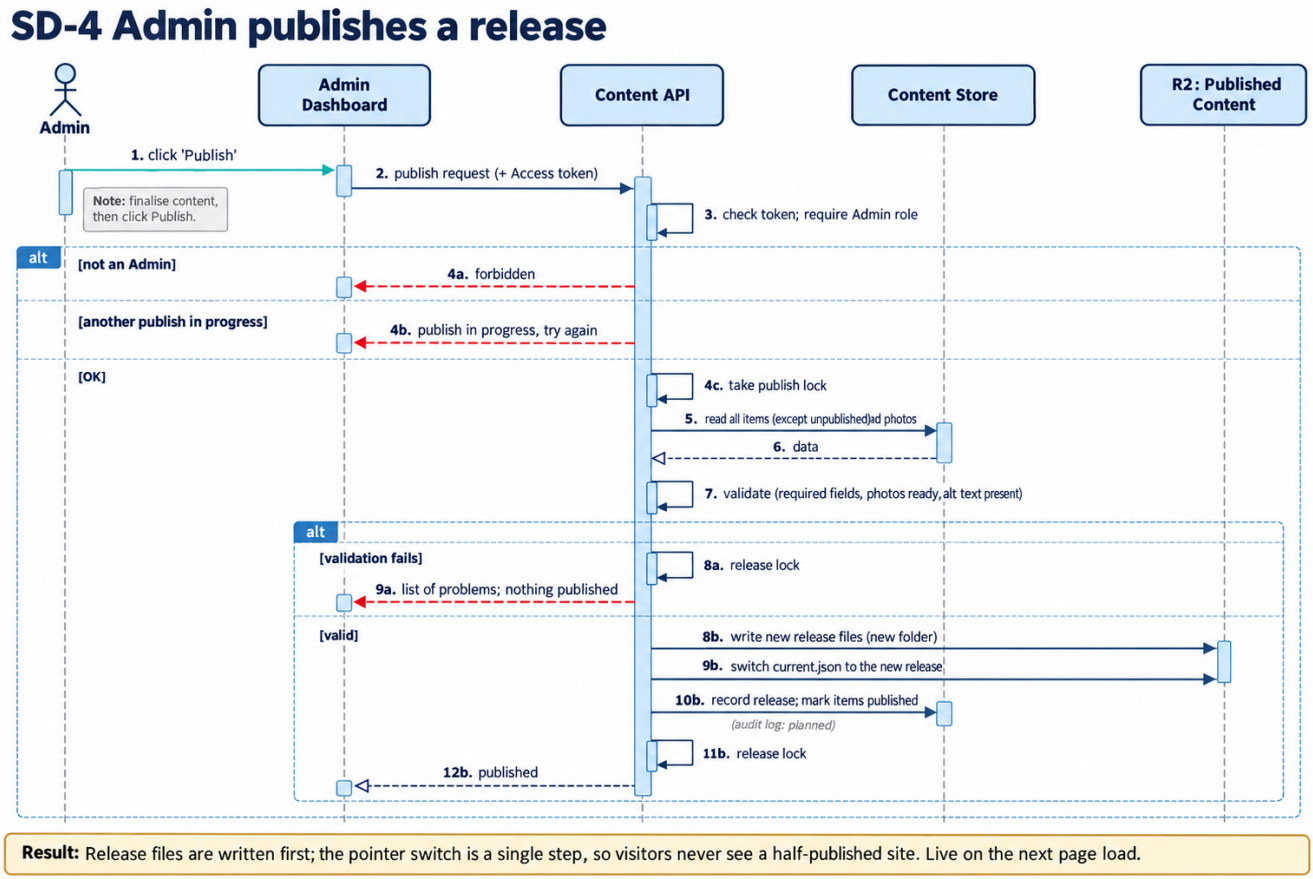


Figure 7: SD-4 Admin publishes a release

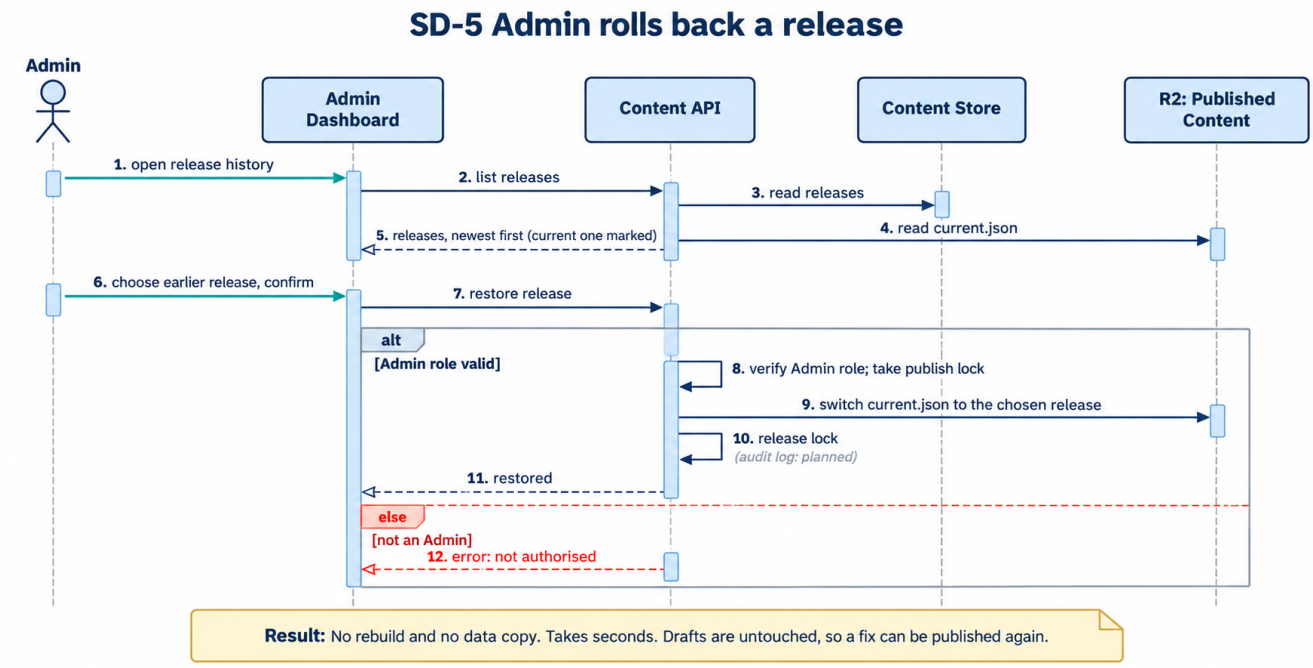


Figure 8: SD-5 Admin rolls back a release

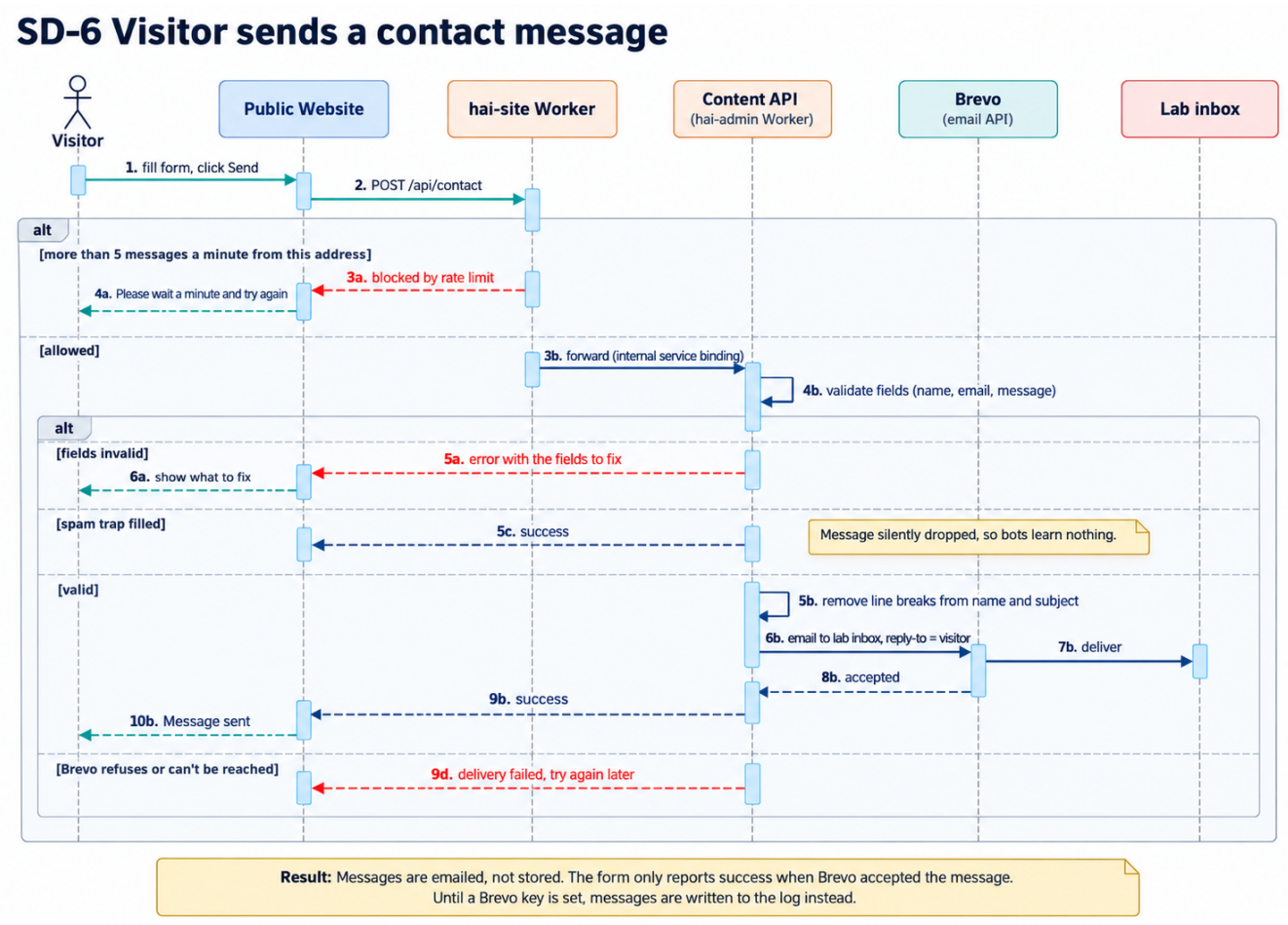


Figure 9: SD-6 Visitor sends a contact message

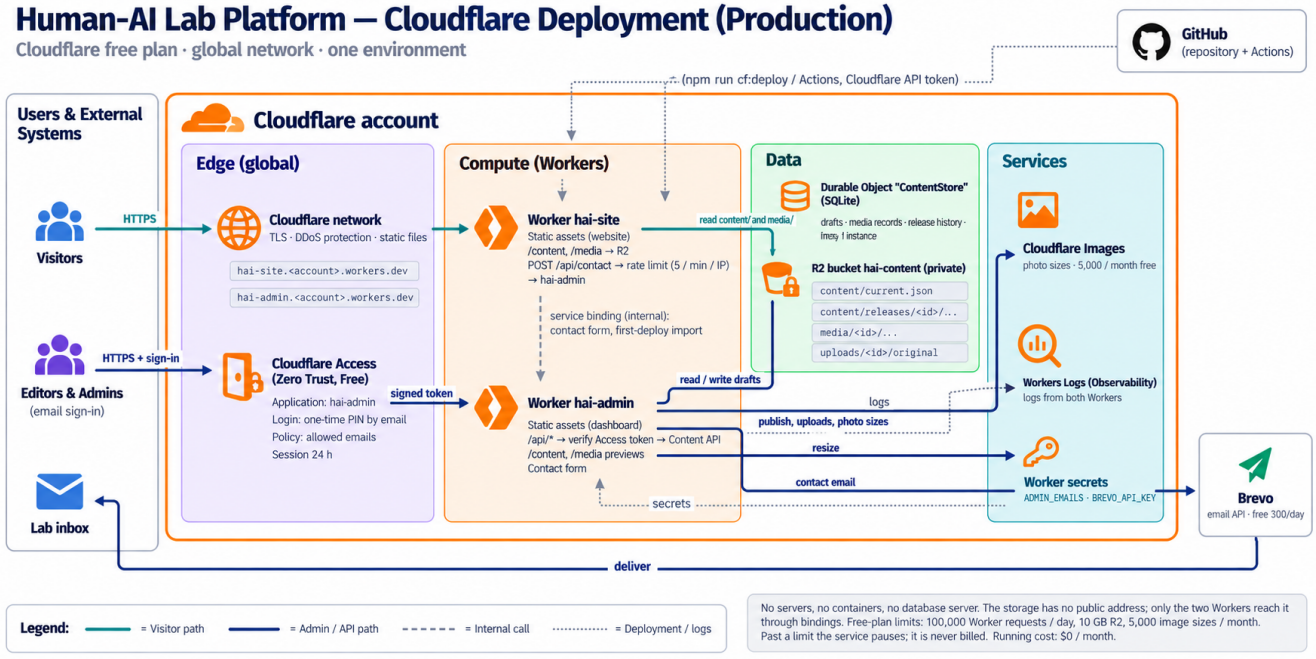


Figure 10: Cloudflare deployment

- **Where it runs:** on Cloudflare's global network, so pages are served from a location near each visitor, including in Sri Lanka. There is no region to choose and no server to maintain.
- **What is deployed:** two Workers (`hai-site` for the public website, `hai-admin` for the dashboard and the Content API), one Durable Object for drafts, one R2 bucket for releases and photos, Cloudflare Images for photo sizes, and Cloudflare Access for sign-in.
- **One environment:** production. Changes are tried locally first with `npm run dev:local`, which runs the whole platform on a laptop without a Cloudflare account.
- **Deployment:** one command, `npm run cf:deploy`, or the **Deploy to Cloudflare** workflow in GitHub. The configuration is kept in the repository; the admin list and the email key are stored as secrets, never in the repository. A newcomer's guide is in `cloudflare/README.md`.
- **Domains:** the site currently runs at `workers.dev` addresses. For the lab's own address, the domain must be managed by Cloudflare: either a lab-owned domain, or `humanai.cse.mrt.ac.lk` delegated to Cloudflare by University IT.

Running cost

	Free allowance	Lab usage
Worker requests	100,000 a day	a small fraction (static pages don't count)
Content Store (Durable Object)	100,000 requests a day, 5 GB	a small fraction
R2 storage	10 GB	well under 1 GB of photos
Cloudflare Images	5,000 photo sizes a month	about 3 per photo uploaded
Cloudflare Access	50 people	the lab's editors and admins
Brevo email	300 emails a day	contact-form messages
Total		**$0 a month**

Past a free limit, the affected service pauses until the next day or month; the free plan never bills for it. Usage alerts can be set up to warn before that happens.

---

7. Security

Figure 11 — Trust zones and controls C1–C10.

In short:

- **One way in.** Every request arrives through Cloudflare's network, which absorbs attacks such as floods. The storage has no public address; only the two Workers can reach it.
- **Invite-only dashboard.** Cloudflare Access lets in only the email addresses on its list, with a code emailed each time. No passwords are stored anywhere. The API checks every sign-in token itself (signature, sender, intended application, expiry) and checks the role for every action.
- **Uploads are treated as hostile.** File type is checked from the file's contents, size limits apply, and every photo is re-encoded with location and camera data removed.
- **The public site only reads published files.** Drafts and original uploads are never served to visitors.
- **Contact form is rate-limited and spam-checked.** At most 5 messages a minute from one address, and a hidden spam trap.
- **No secrets in the repository.** The repository is public; the admin list and the email key are stored as deployment secrets.
- **Recoverable.** Every release is kept, so the site can return to any earlier version in seconds.

---

8. Key design decisions

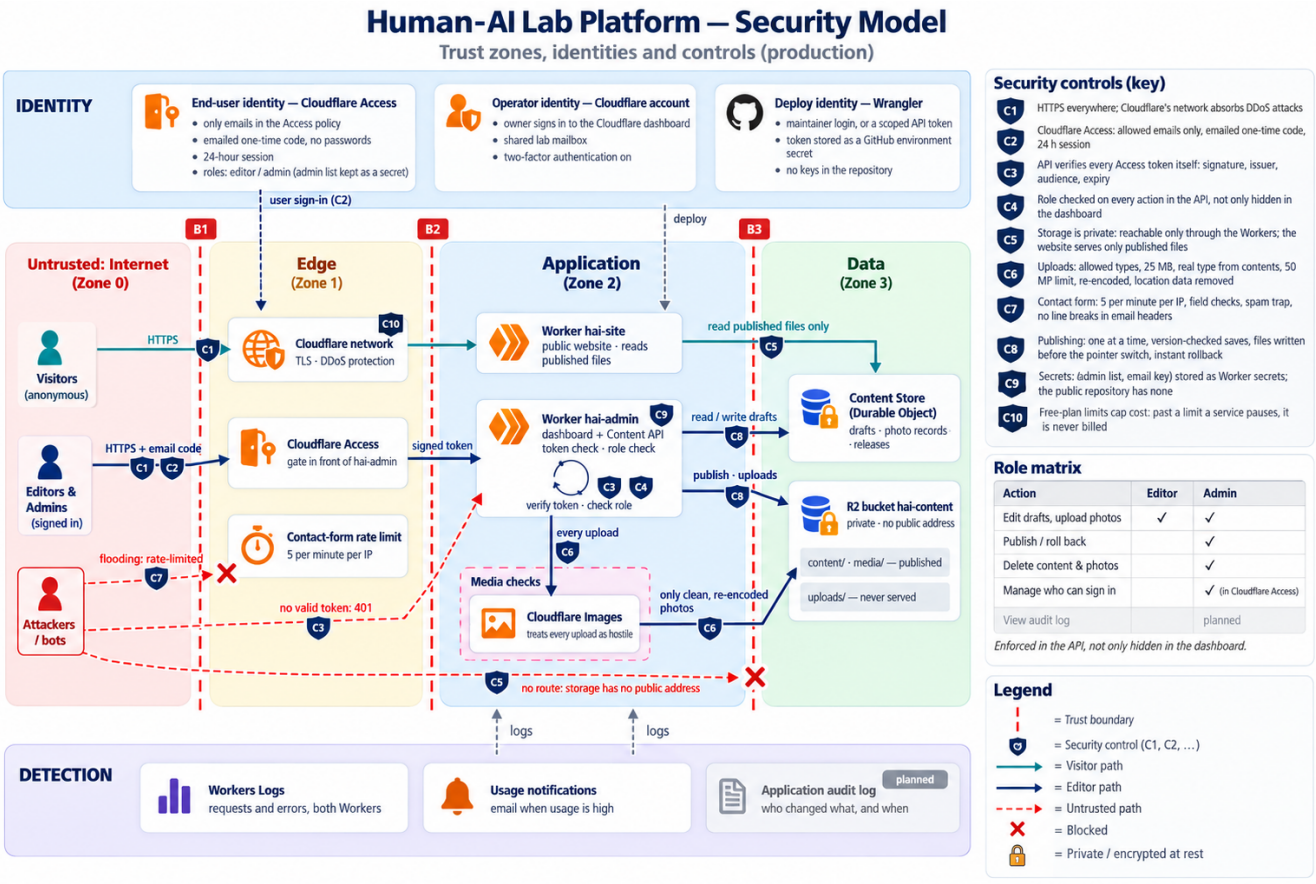


Figure 11: Security model

Decision	Chosen over	Reason
Publish frozen snapshots; visitors read only those	Site reads the drafts database live	Public site stays up even if the dashboard fails; instant rollback
Build a small custom dashboard	Hosted CMS (Sanity, Contentful)	Data stays in the lab's account; no per-seat fees; same technology as the website
Cloudflare free plan	AWS (planned in version 0.2, ≈ $65–75 a month)	$0 running cost; global network close to visitors; nothing to patch; one-command deployment a newcomer can run
TypeScript Content API in a Worker	Java / Spring Boot (built first)	Runs on the free plan without containers; one language across website, dashboard and API; same API contract and tests, ported
Durable Object with SQLite for drafts	DynamoDB, a relational database server	Free; one instance makes version checks and the publish lock exact
Cloudflare Access for sign-in	Amazon Cognito, self-built accounts	Free for up to 50 people; no passwords to store; invite-only by email
Brevo for contact email	Amazon SES, SMTP	Cloudflare's servers can't send over SMTP; Brevo works over HTTPS and is free at this volume
One production environment	Separate staging and production	The whole platform runs locally for testing; a second environment adds cost and upkeep without much benefit at this size

The main trade-offs: - **Free-plan limits.** They are far above the lab's needs, but a large flood of requests could pause the site's content until the next day. A custom domain on Cloudflare adds free firewall rules to reduce this. - **One provider.** The site depends on a single provider. The API follows an open contract (`contracts/openapi.yaml`) and the data is plain files and SQLite, so it could move again. - **The Java codebase is no longer the deployed one.** It remains in the repository for reference.

---

9. Roadmap

The platform is built and running on Cloudflare. The public site stays on Netlify until the dashboard is in regular use and the lab's domain is ready, so visitors see no disruption. Weeks below count from **W1 = 28 September 2026** and are **proposed, to be confirmed**.

Figure 12 — Project plan. Phases (coloured bars) run in dependency order; red diamonds are milestones; the bottom rows are admin work that runs in parallel.

9.1 Milestones

Milestone	When	What it proves
M1 Platform live on Cloudflare	**Done** (27 Sep 2026)	Website, dashboard and API run on Cloudflare; the existing content was imported and published
M2 Editors publish without developers	W4	Lab members update the site on their own
M3 Go-live on the lab's domain	W8	Visitors are served from Cloudflare; Netlify is retired after a fallback period

Milestone	When	What it proves
M4 Pages visible to search engines	W11	News and project pages show up properly in search results and link previews

9.2 What is done

- The Content API, rewritten in TypeScript for Cloudflare: drafts, publishing, rollback and photo uploads, following the same contract and tests as the Java version.
- The existing site content (3 news articles, 5 publications, 4 research themes, 20 people) imported and published; the published files match the site's original data exactly.
- The dashboard, live behind Cloudflare Access, with editor and admin roles.
- One-command deployment, a GitHub workflow and a newcomer's guide.

9.3 Remaining phases

P1 — Finish setup (W1). Connect the contact form to Brevo, invite the editors in Cloudflare Access, and confirm that phone photos come out upright.

P2 — Media migration (W1–W3). Move the 26 images currently hotlinked from third-party hosts (snipboard.io, Cloudinary) into the lab's own media library. Scheduled early because hotlinked images can disappear at any time.

P3 — Editor onboarding (W2–W4). Short training for editors and admins, then editors publish on their own. *Outcome: M2.*

P4 — Go-live (W5–W8). Add the lab's domain to Cloudflare (or have `humanai.cse.mrt.ac.lk` delegated), attach it to the website, and switch on the free firewall rules and bot protection. DNS is switched to Cloudflare in W8. Netlify is kept running as a fallback for two weeks and retired afterwards. *Outcome: M3.*

P5 — Dashboard features (W5–W12). Projects in the dashboard, page slots and placements, a user list and the audit log, then video.

P6 — Search engine visibility (W9–W11). Publishing also produces a static HTML version of every page, and a sitemap, so search engines and link previews can read the content. *Outcome: M4.*

Later. Features for when they are needed: server-rendered pages, sign-in with university accounts through Cloudflare Access, scheduled publishing, search, and regular exports of the drafts as a backup.

9.4 Cut-over and fallback

The switch happens in **W8**. For the next two weeks, the Netlify site is left running untouched. If a serious problem appears, DNS is pointed back to Netlify, and visitors return to the old site within minutes. Netlify is only retired once the new site has run cleanly through this window.

9.5 Parallel tracks (administrative work)

These tasks involve the university and the department rather than code. Some have long lead times, so they start now.

Task	Weeks	Why it matters
Ask University IT to delegate `humanai.cse.mrt.ac.lk` to Cloudflare	W1–W6 · **start now**	Needed for the production address; depends on another team's schedule
Register the lab domain and choose the primary (canonical) one	W1–W2	Decides the site's permanent address, and is the fallback if delegation is slow

Task	Weeks	Why it matters
Move the Cloudflare account to a shared lab mailbox and add a second account admin	W1–W2	The account must not depend on one person
Editor onboarding and training	W2–W4	Editors must be confident before go-live

---

How to deploy and run the platform is described in `cloudflare/README.md`. The Technical Design Document describes the earlier AWS design and has not yet been updated.

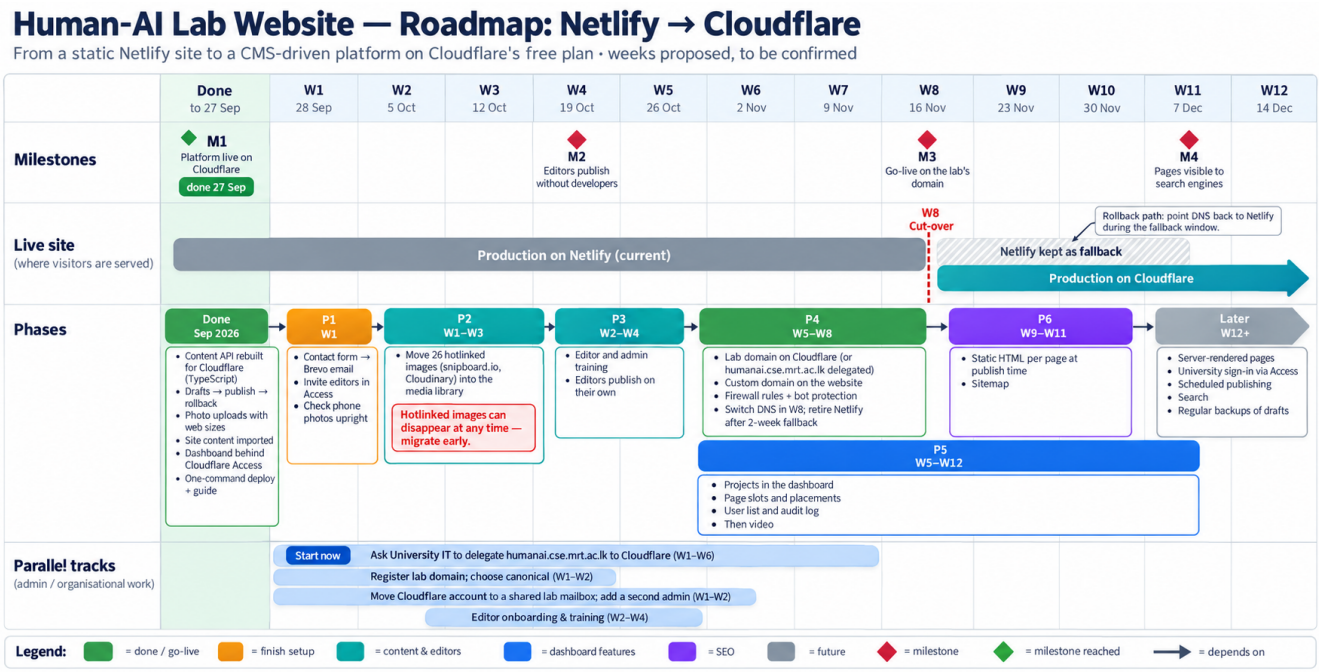


Figure 12: Project roadmap: Netlify to Cloudflare